

# Liqing Accounting Software V1.0 - Source Code Document

Copyright owner: Xiamen Chizhuo Culture Media Co., Ltd.

Development completed: 2026-04-30

Source scope: client/lib, server/src, server/prisma

Source files: 63

Total source lines: 18956

Submission rule: total source lines exceed 3000; provide first 1500 and last 1500 lines.

Liqing Accounting Software V1.0 FIRST PAGE 01

(50 source lines on this page)

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'core/router/app_router.dart';
import 'core/providers/preferences_provider.dart';
import 'core/theme/app_theme.dart';

class OneKeepApp extends ConsumerWidget {
  const OneKeepApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(routerProvider);
    final preferences = ref.watch(preferencesProvider);

    return MaterialApp.router(
      title: '___',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      darkTheme: AppTheme.dark,
      themeMode: preferences.themeMode,
      routerConfig: router,
    );
  }
}

class FeatureFlags {
  const FeatureFlags._();

  static const aiFeaturesEnabled = false;
}

/// ___ AI ___
/// API Key _____ - AI _____
class AiDefaults {
  static const String baseUrl = 'https://api.deepseek.com/v1';

  static const String apiKey = '';

  static const String modelName = 'deepseek-v4-flash';
}

import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class ApiClient {
  static const _tokenKey = 'jwt_token';
  static final _storage = FlutterSecureStorage();

  late final Dio dio;
```

```
ApiClient({String baseUrl = 'http://localhost:3000'}) {
  dio = Dio(
    BaseOptions(
```

-----

Liqing Accounting Software V1.0      FIRST PAGE 02

=====

(50 source lines on this page)

```
    baseUrl: baseUrl,
    connectTimeout: const Duration(seconds: 10),
    receiveTimeout: const Duration(seconds: 10),
  ),
);

dio.interceptors.add(
  InterceptorsWrapper(
    onRequest: (options, handler) async {
      final token = await _storage.read(key: _tokenKey);
      if (token != null) {
        options.headers['Authorization'] = 'Bearer $token';
      }
      final method = options.method.toUpperCase();
      if (options.data == null &&
        (method == 'GET' || method == 'DELETE' || method == 'HEAD')) {
        options.contentType = null;
        options.headers.remove('Content-Type');
      } else if (options.data != null && options.contentType == null) {
        options.contentType = Headers.jsonContentType;
        options.headers['Content-Type'] = Headers.jsonContentType;
      }
      handler.next(options);
    },
    onError: (error, handler) {
      if (error.response?.statusCode == 401) {
        // Token ----- token
        _storage.delete(key: _tokenKey);
      }
      handler.next(error);
    },
  ),
);

static Future<void> saveToken(String token) =>
  _storage.write(key: _tokenKey, value: token);

static Future<void> clearToken() => _storage.delete(key: _tokenKey);

static Future<String?> getToken() => _storage.read(key: _tokenKey);

static String readableError(Object error, {String fallback = '____'}) {
  if (error is DioException) {
    final data = error.response?.data;
    if (data is Map<String, dynamic>) {
      final message = data['message'] ?? data['error'];
      if (message is String && message.trim().isNotEmpty) {
        return message.trim();
      }
    }
  }
}
```

-----

=====

(50 source lines on this page)

```

        }
        if (data is String && data.trim().isNotEmpty) {
            return data.trim();
        }
        final statusCode = error.response?.statusCode;
        if (statusCode != null) {
            return '$fallback_HTTP $statusCode_';
        }
    }
    return fallback;
}
}
import 'package:flutter/foundation.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../network/api_client.dart';

const _defaultApiBaseUrl = 'https://liqing.eatdesk.net';
const _localApiBaseUrl = 'http://192.168.10.231:3002';

final apiClientProvider = Provider<ApiClient>((ref) {
    return ApiClient(baseUrl: _resolveApiBaseUrl());
});

String _resolveApiBaseUrl() {
    const override = String.fromEnvironment('API_BASE_URL');
    if (override.isNotEmpty) return override;

    if (kIsWeb) {
        final origin = Uri.base.origin;
        final isLocalHost =
            Uri.base.host == 'localhost' || Uri.base.host == '127.0.0.1';
        if (kReleaseMode || !isLocalHost) {
            return origin;
        }
    }

    if (kReleaseMode) return _defaultApiBaseUrl;

    return _localApiBaseUrl;
}
import 'dart:convert';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import '../network/api_client.dart';
import '../..../shared/models/models.dart';
import 'api_provider.dart';

```

/// ----

=====

(50 source lines on this page)

```

enum AuthStatus { unknown, authenticated, unauthenticated }

const Object _unset = Object();

class AuthState {
  final AuthStatus status;
  final UserInfo? user;
  final bool isLoading;
  final bool isSendingCode;
  final String? error;

  const AuthState({
    this.status = AuthStatus.unknown,
    this.user,
    this.isLoading = false,
    this.isSendingCode = false,
    this.error,
  });

  AuthState copyWith({
    AuthStatus? status,
    Object? user = _unset,
    bool? isLoading,
    bool? isSendingCode,
    Object? error = _unset,
  }) => AuthState(
    status: status ?? this.status,
    user: identical(user, _unset) ? this.user : user as UserInfo?,
    isLoading: isLoading ?? this.isLoading,
    isSendingCode: isSendingCode ?? this.isSendingCode,
    error: identical(error, _unset) ? this.error : error as String?,
  );
}

class AuthNotifier extends StateNotifier<AuthState> {
  final ApiClient _api;
  static const _storage = FlutterSecureStorage();
  static const _currentUserKey = 'auth_current_user';

  AuthNotifier(this._api) : super(const AuthState()) {
    _checkAuth();
  }

  Future<void> _checkAuth() async {
    final token = await _api.dio.getToken();
    if (token == null) {
      await _storage.delete(key: _currentUserKey);
      state = state.copyWith(status: AuthStatus.unauthenticated, user: null);
      return;
    }
  }
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 05

=====

(50 source lines on this page)

```

try {
  final res = await _api.dio.get('/api/auth/me');
  final data = res.data as Map<String, dynamic>;
  final user = UserInfo.fromJson(

```

```

        Map<String, dynamic>.from(data['user'] as Map<String, dynamic>),
    );
    await _writeStoredUser(user);
    state = state.copyWith(status: AuthStatus.authenticated, user: user);
  } catch (_) {
    await ApiClient.clearToken();
    await _storage.delete(key: _currentUserKey);
    state = state.copyWith(status: AuthStatus.unauthenticated, user: null);
  }
}

```

```

Future<void> login(String identifier, String password) async {
  state = state.copyWith(isLoading: true, error: null);
  try {
    final res = await _api.dio.post(
      '/api/auth/login',
      data: {'identifier': identifier, 'password': password},
    );
    final data = res.data as Map<String, dynamic>;
    final user = Map<String, dynamic>.from(
      data['user'] as Map<String, dynamic>,
    );
    user['name'] ??= user['displayName'];
    final userInfo = UserInfo.fromJson(user);
    await ApiClient.saveToken(data['token'] as String);
    await _writeStoredUser(userInfo);
    state = state.copyWith(
      status: AuthStatus.authenticated,
      user: userInfo,
      isLoading: false,
    );
  } catch (e) {
    state = state.copyWith(
      isLoading: false,
      error: ApiClient.readableError(e, fallback: '____'),
    );
  }
}

```

```

Future<void> register(
  String username,
  String email,
  String password,
  String displayName, [
  String? code,

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 06

=====

(50 source lines on this page)

```

]) async {
  state = state.copyWith(isLoading: true, error: null);
  try {
    final payload = <String, dynamic>{
      'username': username,
      'email': email,
      'password': password,
      'displayName': displayName,
    };
    if (code != null && code.trim().isEmpty) {

```

```

    payload['code'] = code.trim();
  }

  final res = await _api.dio.post('/api/auth/register', data: payload);
  final data = res.data as Map<String, dynamic>;
  final user = Map<String, dynamic>.from(
    data['user'] as Map<String, dynamic>,
  );
  user['name'] ??= user['displayName'];
  final userInfo = UserInfo.fromJson(user);
  await ApiClient.saveToken(data['token'] as String);
  await _writeStoredUser(userInfo);
  state = state.copyWith(
    status: AuthStatus.authenticated,
    user: userInfo,
    isLoading: false,
  );
} catch (e) {
  state = state.copyWith(
    isLoading: false,
    error: ApiClient.readableError(e, fallback: '____'),
  );
}
}

/// -----
Future<bool> sendCode(String email) async {
  state = state.copyWith(isSendingCode: true, error: null);
  try {
    await _api.dio.post('/api/auth/send-code', data: {'email': email});
    state = state.copyWith(isSendingCode: false);
    return true;
  } catch (e) {
    state = state.copyWith(
      isSendingCode: false,
      error: ApiClient.readableError(e, fallback: '____'),
    );
    return false;
  }
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 07

=====

(50 source lines on this page)

```

/// -----/-----
Future<void> verifyCode(String email, String code) async {
  state = state.copyWith(isLoading: true, error: null);
  try {
    final res = await _api.dio.post(
      '/api/auth/verify-code',
      data: {'email': email, 'code': code},
    );
    final data = res.data as Map<String, dynamic>;
    final user = Map<String, dynamic>.from(
      data['user'] as Map<String, dynamic>,
    );
    user['name'] ??= user['displayName'];
    final userInfo = UserInfo.fromJson(user);
  }
}

```

```

    await ApiClient.saveToken(data['token'] as String);
    await _writeStoredUser(userInfo);
    state = state.copyWith(
      status: AuthStatus.authenticated,
      user: userInfo,
      isLoading: false,
    );
  } catch (e) {
    state = state.copyWith(
      isLoading: false,
      error: ApiClient.readableError(e, fallback: '____'),
    );
  }
}

Future<void> logout() async {
  await ApiClient.clearToken();
  await _storage.delete(key: _currentUserKey);
  state = const AuthState(status: AuthStatus.unauthenticated);
}

void updateLocalUser({String? name}) {
  if (state.user == null) return;
  final user = state.user!.copyWith(name: name);
  state = state.copyWith(user: user);
  _writeStoredUser(user);
}

void clearError() {
  state = state.copyWith(error: null);
}

void setError(String message) {
  state = state.copyWith(error: message);
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 08

=====

(50 source lines on this page)

```

Future<void> _writeStoredUser(UserInfo user) {
  return _storage.write(
    key: _currentUserKey,
    value: jsonEncode({
      'id': user.id,
      'username': user.username,
      'name': user.name,
      'email': user.email,
    })),
  );
}

final authProvider = StateNotifierProvider<AuthNotifier, AuthState>((ref) {
  return AuthNotifier(ref.read(apiClientProvider));
});
import 'dart:convert';

import 'package:dio/dio.dart';

```

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../..../core/providers/api_provider.dart';
import '../..../core/providers/data_providers.dart';
import '../..../core/providers/preferences_provider.dart';
import '../..../core/services/ai_endpoint_utils.dart';
import '../..../shared/models/models.dart';
```

```
/// -----
class ChatMessage {
  final String id;
  final String content;
  final bool isUser;
  final DateTime timestamp;
  final List<ParsedTransaction>? transactions;

  const ChatMessage({
    required this.id,
    required this.content,
    required this.isUser,
    required this.timestamp,
    this.transactions,
  });

  ChatMessage copyWith({
    String? id,
    String? content,
    bool? isUser,
    DateTime? timestamp,
    List<ParsedTransaction>? transactions,
  }) {
    return ChatMessage(
      id: id ?? this.id,
      content: content ?? this.content,
      isUser: isUser ?? this.isUser,
      timestamp: timestamp ?? this.timestamp,
      transactions: transactions ?? this.transactions,
    );
  }
}
```

-----

Liqing Accounting Software V1.0      FIRST PAGE 09

=====

(50 source lines on this page)

```
bool clearTransactions = false,
}) => ChatMessage(
  id: id ?? this.id,
  content: content ?? this.content,
  isUser: isUser ?? this.isUser,
  timestamp: timestamp ?? this.timestamp,
  transactions: clearTransactions
    ? null
    : (transactions ?? this.transactions),
);
}
```

```
/// AI -----
class ParsedTransaction {
  final String title;
  final double amount;
  final String direction;
  final String? categoryName;
  final String? note;

  const ParsedTransaction({
    required this.title,
    required this.amount,
    required this.direction,
    this.categoryName,
    this.note,
  });
}
```

```

        this.note,
    });

    factory ParsedTransaction.fromJson(Map<String, dynamic> json) =>
        ParsedTransaction(
            title: json['title'] as String? ?? '___',
            amount: (json['amount'] as num?)?.toDouble() ?? 0,
            direction: json['direction'] as String? ?? 'expense',
            categoryName: json['categoryName'] as String?,
            note: json['note'] as String?,
        );
}

/// ----
class ChatState {
    final List<ChatMessage> messages;
    final bool isLoading;
    final String? error;
    final Set<String> confirmingMessageIds;
    final Set<String> confirmedMessageIds;

    const ChatState({
        this.messages = const [],
        this.isLoading = false,
        this.error,
    })

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 10

=====

(50 source lines on this page)

```

        this.confirmingMessageIds = const {},
        this.confirmedMessageIds = const {},
    });

    ChatState copyWith({
        List<ChatMessage>? messages,
        bool? isLoading,
        String? error,
        Set<String>? confirmingMessageIds,
        Set<String>? confirmedMessageIds,
    }) => ChatState(
        messages: messages ?? this.messages,
        isLoading: isLoading ?? this.isLoading,
        error: error,
        confirmingMessageIds: confirmingMessageIds ?? this.confirmingMessageIds,
        confirmedMessageIds: confirmedMessageIds ?? this.confirmedMessageIds,
    );

    bool isTransactionConfirming(String messageId) =>
        confirmingMessageIds.contains(messageId);

    bool isTransactionConfirmed(String messageId) =>
        confirmedMessageIds.contains(messageId);
}

class ChatNotifier extends StateNotifier<ChatState> {
    ChatNotifier(this._ref) : super(const ChatState());

    final Ref _ref;
}

```

```

/// ----- AI -----
Future<void> sendMessage(String content) async {
  if (content.trim().isEmpty) return;

  final userMessage = ChatMessage(
    id: _uuid(),
    content: content.trim(),
    isUser: true,
    timestamp: DateTime.now(),
  );

  final updatedMessages = [...state.messages, userMessage];
  state = state.copyWith(
    messages: updatedMessages,
    isLoading: true,
    error: null,
  );

  try {
    final categories = _ref.read(categoriesProvider).valueOrNull ?? [];

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 11

=====

(50 source lines on this page)

```

    final aiResponse = await _callAiForBookkeeping(content, categories);

    final assistantMessage = ChatMessage(
      id: _uuid(),
      content: aiResponse['reply'] as String? ?? '-----',
      isUser: false,
      timestamp: DateTime.now(),
      transactions: (aiResponse['transactions'] as List<dynamic>?)
        ?.map((e) => ParsedTransaction.fromJson(e as Map<String, dynamic>))
        .toList(),
    );

    state = state.copyWith(
      messages: [...state.messages, assistantMessage],
      isLoading: false,
    );
  } catch (e) {
    // -----
    final errorMsg = ChatMessage(
      id: _uuid(),
      content: e.toString().replaceFirst('Exception: ', ''),
      isUser: false,
      timestamp: DateTime.now(),
    );
    state = state.copyWith(
      messages: [...state.messages, errorMsg],
      isLoading: false,
      error: e.toString(),
    );
  }
}

/// ----- AI -----
Future<void> confirmTransactions(
  String messageId,

```

```

List<ParsedTransaction> transactions,
) async {
  if (state.isTransactionConfirming(messageId) ||
      state.isTransactionConfirmed(messageId)) {
    return;
  }

  final message = state.messages
    .where((item) => item.id == messageId)
    .firstOrNull;
  if (message == null ||
      message.transactions == null ||
      message.transactions!.isEmpty) {
    return;
  }

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 12

=====

(50 source lines on this page)

```

state = state.copyWith(
  confirmingMessageIds: {...state.confirmingMessageIds, messageId},
  error: null,
);

final categories = _ref.read(categoriesProvider).valueOrNull ?? [];
var createdCount = 0;

for (final tx in transactions) {
  String? categoryId;
  if (tx.categoryName != null) {
    final match = categories
      .where(
        (c) =>
          c.name == tx.categoryName ||
          c.name.contains(tx.categoryName!),
      )
      .firstOrNull;
    categoryId = match?.id;
  }
  if (categoryId == null) {
    final fallback = categories
      .where((c) => c.type == tx.direction)
      .firstOrNull;
    categoryId = fallback?.id;
  }
  if (categoryId == null) continue;

  try {
    final api = _ref.read(apiClientProvider);
    await api.dio.post(
      '/api/transactions',
      data: {
        'title': tx.title,
        'amount': tx.amount,
        'direction': tx.direction,
        'categoryId': categoryId,
        'occurredAt': DateTime.now().toUtc().toIso8601String(),
        if (tx.note != null) 'note': tx.note,
      },
    );
  } catch (e) {
    // Handle error
  }
}

```

```

    },
    );
    createdCount += 1;
} catch (_) {
    // -----
}
}

_ref.read(homeProvider.notifier).load();
_ref.read(billsProvider.notifier).load();

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 13

=====

(50 source lines on this page)

```

final updatedMessages = createdCount > 0
    ? state.messages
      .map(
        (item) => item.id == messageId
          ? item.copyWith(clearTransactions: true)
          : item,
      )
      .toList()
    : state.messages;

final confirmMsg = ChatMessage(
  id: _uuid(),
  content: createdCount > 0
    ? '----- $createdCount ----'
    : '-----',
  isUser: false,
  timestamp: DateTime.now(),
);

state = state.copyWith(
  messages: [...updatedMessages, confirmMsg],
  confirmingMessageIds: state.confirmingMessageIds
    .where((id) => id != messageId)
    .toSet(),
  confirmedMessageIds: createdCount > 0
    ? {...state.confirmedMessageIds, messageId}
    : state.confirmedMessageIds,
);
}

/// -----
void clearChat() {
  state = const ChatState();
}

Future<Map<String, dynamic>> _callAiForBookkeeping(
  String userMessage,
  List<Category> categories,
) async {
  final prefs = _ref.read(preferencesProvider);
  if (!prefs.hasAiConfigured) {
    throw Exception('----- AI ----- AI --');
  }
}

```

```

final modelName = prefs.aiModelName.isNotEmpty
    ? prefs.aiModelName
    : 'gpt-4o-mini';

try {

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 14

=====

(50 source lines on this page)

```

final categoryList = categories
    .map((c) => ' - ${c.name}_${c.type == 'expense' ? '__' : '__'}_')
    .join('\n');

```

```

final systemPrompt =
    '-----

```

-----

\$categoryList

```

JSON ----- markdown -----
{"reply": "-----", "transactions": [{"title": "-----", "amount": -----, "direction": "ex
pense_income", "categoryName": "-----", "note": "-----"}]}

```

```

-----
1. amount -----
2. direction ___ "expense" _ "income"
3. categoryName -----
4. ----- transactions -----
5. ----- reply ----- transactions -----
6. ----- JSON ----- markdown ----- '---';

```

```

final dio = Dio();
final response = await dio.post(
  aiChatCompletionsUrl(prefs.aiApiBaseUrl),
  data: {
    'model': modelName,
    'messages': [
      {'role': 'system', 'content': systemPrompt},
      // ----- AI ----- <think/> -----
      ...state.messages
        .take(20)
        .map(
          (m) => {
            'role': m.isUser ? 'user' : 'assistant',
            'content': m.isUser
              ? m.content
              : _stripThinkTags(m.content),
          },
        ),
    ],
    'temperature': 0.2,
  },
  options: Options(
    headers: {
      'Authorization': 'Bearer ${prefs.aiApiKey}',
      'Content-Type': 'application/json',
    },
    sendTimeout: const Duration(seconds: 30),
    receiveTimeout: const Duration(seconds: 60),
  ),
);

```

),

-----  
Liqing Accounting Software V1.0 FIRST PAGE 15

=====  
(50 source lines on this page)

```
);

// __ AI __
final body = response.data as Map<String, dynamic>;
final choices = body['choices'] as List<dynamic>;
if (choices == null || choices.isEmpty) {
  throw Exception('AI -----');
}

// __ finish_reason
final finishReason = choices[0]['finish_reason'] as String? ?? 'stop';
if (finishReason == 'content_filter' || finishReason == 'length') {
  throw Exception('AI -----$finishReason-----');
}

var content = choices[0]['message']?['content'] as String?;

// __ content _ null _ reasoning_content _
if (content == null || content.trim().isEmpty) {
  final reasoning =
    choices[0]['message']?['reasoning_content'] as String?;
  if (reasoning != null && reasoning.trim().isNotEmpty) {
    // -----
    // ----- JSON
    content = reasoning;
  } else {
    throw Exception(
      '___ "$modelName" ----- _ AI -----',
    );
  }
}

// ----- JSON -----
Map<String, dynamic> result;
try {
  final jsonStr = _extractJson(content);
  result = jsonDecode(jsonStr) as Map<String, dynamic>;
} catch (_) {
  result = {'reply': content, 'transactions': <dynamic>[]};
}

return result;
} on DioException catch (e) {
  String errorMsg;
  if (e.type == DioExceptionType.connectionTimeout ||
    e.type == DioExceptionType.sendTimeout ||
    e.type == DioExceptionType.receiveTimeout) {
    errorMsg = '-----';
  } else if (e.response?.statusCode != null) {
    final code = e.response!.statusCode!;
```

-----  
Liqing Accounting Software V1.0 FIRST PAGE 16

=====

(50 source lines on this page)

```
switch (code) {
    case 401:
        errorMsg = 'API Key ----- AI -----';
        break;
    case 403:
        errorMsg = 'API -----';
        break;
    case 404:
        errorMsg = 'API ----- "$modelName" -----';
        break;
    case 429:
        errorMsg = '-----';
        break;
    case 500:
    case 502:
    case 503:
    case 504:
        errorMsg = 'AI -----HTTP $code-----';
        break;
    default:
        errorMsg = 'AI -----HTTP $code_';
}
} else {
    errorMsg = '----- AI ----- API --';
}
throw Exception(errorMsg);
} catch (e) {
    throw Exception('AI -----${e.toString().split('\n').first}');
}
}

String _extractJson(String text) {
    // ----- <think...</think*> -----
    var cleaned = text.replaceAll(RegExp(r'<think[\s\S]*?</think\s*>'), '');
    // -- <think... -----
    cleaned = cleaned.replaceAll(RegExp(r'<think[\s\S]*'), '');
    cleaned = cleaned.trim();

    // ---- markdown ----- JSON
    final codeBlockRegex = RegExp(r'```(?:json)?\s*([\s\S]*?)\s*```');
    final match = codeBlockRegex.firstMatch(cleaned);
    if (match != null) {
        return match.group(1)!.trim();
    }

    // ----- { ... } JSON --
    final braceStart = cleaned.indexOf('{');
    final braceEnd = cleaned.lastIndexOf('}');
    if (braceStart != -1 && braceEnd != -1 && braceEnd > braceStart) {
        return cleaned.substring(braceStart, braceEnd + 1);
    }
}
```

=====

(50 source lines on this page)

}

```

    return cleaned;
}

/// ___ AI ___ `<think/>` -----
String _stripThinkTags(String text) {
    var cleaned = text.replaceAll(RegExp(r'<think[\s\S]*?</think\s*>'), '');
    cleaned = cleaned.replaceAll(RegExp(r'<think[\s\S]*'), '');
    return cleaned.trim();
}

String _uuid() => DateTime.now().microsecondsSinceEpoch.toString();
}

final chatProvider = StateNotifierProvider<ChatNotifier, ChatState>((ref) {
    return ChatNotifier(ref);
});
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import '../network/api_client.dart';
import '../shared/models/models.dart';
import 'api_provider.dart';
import 'auth_provider.dart';

// ===== Home =====

class HomeState {
    final HomeSummary? summary;
    final bool isLoading;
    final String? error;

    const HomeState({this.summary, this.isLoading = false, this.error});
}

class HomeNotifier extends StateNotifier<HomeState> {
    final ApiClient _api;

    HomeNotifier(this._api) : super(const HomeState());

    Future<void> load([String? month]) async {
        state = HomeState(isLoading: true);
        try {
            final m = month ?? DateFormat('yyyy-MM').format(DateTime.now());
            final res = await _api.dio.get('/api/home/summary', queryParameters: {'month': m});
            state = HomeState(summary: HomeSummary.fromJson(res.data as Map<String, dynamic>));
        } catch (e) {
            state = HomeState(error: '____');
        }
    }
}

-----

Liqing Accounting Software V1.0    FIRST PAGE 18
=====
(50 source lines on this page)

}

final homeProvider = StateNotifierProvider<HomeNotifier, HomeState>((ref) {
    return HomeNotifier(ref.read(apiClientProvider));
});

```

```
// ===== Stats =====

class StatsState {
  final StatsOverview? overview;
  final bool isLoading;
  final String? error;

  const StatsState({this.overview, this.isLoading = false, this.error});
}

class StatsNotifier extends StateNotifier<StatsState> {
  final ApiClient _api;

  StatsNotifier(this._api) : super(const StatsState());

  Future<void> load({String? month, String metricType = 'expense'}) async {
    state = StatsState(isLoading: true);
    try {
      final m = month ?? DateFormat('yyyy-MM').format(DateTime.now());
      final res = await _api.dio.get('/api/stats/overview', queryParameters: {
        'month': m,
        'metricType': metricType,
      });
      state = StatsState(overview: StatsOverview.fromJson(res.data as Map<String
, dynamic>));
    } catch (e) {
      state = StatsState(error: '____');
    }
  }
}

final statsProvider = StateNotifierProvider<StatsNotifier, StatsState>((ref) {
  return StatsNotifier(ref.read(apiClientProvider));
});

// ===== Bills =====

class BillsState {
  final List<BillGroup> groups;
  final bool isLoading;
  final bool hasMore;
  final String? nextCursor;
  final String? error;

  const BillsState({

-----

Liqing Accounting Software V1.0    FIRST PAGE 19
=====
(50 source lines on this page)

    this.groups = const [],
    this.isLoading = false,
    this.hasMore = true,
    this.nextCursor,
    this.error,
  });
}

class BillsNotifier extends StateNotifier<BillsState> {
```

```

final ApiClient _api;

BillsNotifier(this._api) : super(const BillsState());

Future<void> load({
  String? month,
  String filterType = 'all',
  String? query,
}) async {
  state = const BillsState(isLoading: true);
  try {
    final m = month ?? DateFormat('yyyy-MM').format(DateTime.now());
    final res = await _api.dio.get('/api/bills', queryParameters: {
      'month': m,
      'filterType': filterType,
      if (query != null && query.isNotEmpty) 'query': query,
    });
    final data = BillsResponse.fromJson(res.data as Map<String, dynamic>);
    state = BillsState(
      groups: data.groupedBills,
      hasMore: data.nextCursor != null,
      nextCursor: data.nextCursor,
    );
  } catch (e) {
    state = BillsState(error: '____');
  }
}

Future<void> loadMore({
  String? month,
  String filterType = 'all',
  String? query,
}) async {
  if (state.isLoading || !state.hasMore) return;
  state = BillsState(
    groups: state.groups,
    isLoading: true,
    hasMore: state.hasMore,
    nextCursor: state.nextCursor,
  );
  try {

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 20

=====

(50 source lines on this page)

```

    final m = month ?? DateFormat('yyyy-MM').format(DateTime.now());
    final res = await _api.dio.get('/api/bills', queryParameters: {
      'month': m,
      'filterType': filterType,
      if (query != null && query.isNotEmpty) 'query': query,
      if (state.nextCursor != null) 'cursor': state.nextCursor,
    });
    final data = BillsResponse.fromJson(res.data as Map<String, dynamic>);
    final merged = [...state.groups];
    for (final g in data.groupedBills) {
      final idx = merged.indexWhere((e) => e.date == g.date);
      if (idx >= 0) {
        merged[idx] = BillGroup(
          date: g.date,

```

```

        expenseTotal: g.expenseTotal,
        incomeTotal: g.incomeTotal,
        items: [...merged[idx].items, ...g.items],
      );
    } else {
      merged.add(g);
    }
  }
  state = BillsState(
    groups: merged,
    hasMore: data.nextCursor != null,
    nextCursor: data.nextCursor,
  );
} catch (e) {
  state = BillsState(
    groups: state.groups,
    error: '-----',
    hasMore: state.hasMore,
    nextCursor: state.nextCursor,
  );
}
}
}

final billsProvider = StateNotifierProvider<BillsNotifier, BillsState>((ref) {
  return BillsNotifier(ref.read(apiClientProvider));
});

// ===== Categories =====

final categoriesProvider = FutureProvider<List<Category>>((ref) async {
  final authState = ref.watch(authProvider);
  if (authState.status != AuthStatus.authenticated) {
    return const <Category>[];
  }
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 21

=====

(50 source lines on this page)

```

    final api = ref.read(apiClientProvider);
    final res = await api.dio.get('/api/categories');
    return (res.data['categories'] as List)
      .map((e) => Category.fromJson(e as Map<String, dynamic>))
      .toList();
  });
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

import '../constants/ai_defaults.dart';
import 'auth_provider.dart';

const _noPreferenceChange = Object();

class PreferencesState {
  final ThemeMode themeMode;
  final String nickname;
  final int avatarIndex;
}

```

```

final String? avatarImageData;
final String? profileBackgroundImageData;
final String aiApiBaseUrl;
final String aiApiKey;
final String aiModelName;
final bool isLoading;

const PreferencesState({
  this.themeMode = ThemeMode.light, // ----- light
  this.nickname = '',
  this.avatarIndex = 0,
  this.avatarImageData,
  this.profileBackgroundImageData,
  this.aiApiBaseUrl = AiDefaults.baseUrl,
  this.aiApiKey = AiDefaults.apiKey,
  this.aiModelName = AiDefaults.modelName,
  this.isLoading = false,
});

bool get hasAiConfigured => aiApiBaseUrl.isNotEmpty && aiApiKey.isNotEmpty;

PreferencesState copyWith({
  ThemeMode? themeMode,
  String? nickname,
  int? avatarIndex,
  Object? avatarImageData = _noPreferenceChange,
  Object? profileBackgroundImageData = _noPreferenceChange,
  String? aiApiBaseUrl,
  String? aiApiKey,
  String? aiModelName,
  bool? isLoading,

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 22

=====

(50 source lines on this page)

```

    }) {
      return PreferencesState(
        themeMode: themeMode ?? this.themeMode,
        nickname: nickname ?? this.nickname,
        avatarIndex: avatarIndex ?? this.avatarIndex,
        avatarImageData: avatarImageData == _noPreferenceChange
          ? this.avatarImageData
          : avatarImageData as String?,
        profileBackgroundImageData:
          profileBackgroundImageData == _noPreferenceChange
            ? this.profileBackgroundImageData
            : profileBackgroundImageData as String?,
        aiApiBaseUrl: aiApiBaseUrl ?? this.aiApiBaseUrl,
        aiApiKey: aiApiKey ?? this.aiApiKey,
        aiModelName: aiModelName ?? this.aiModelName,
        isLoading: isLoading ?? this.isLoading,
      );
    }
  }

class PreferencesNotifier extends StateNotifier<PreferencesState> {
  PreferencesNotifier(this._ref) : super(const PreferencesState()) {
    _load();
  }
}

```

```

final Ref _ref;
static const _storage = FlutterSecureStorage();
static const _themeModeKey = 'pref_theme_mode';
static const _nicknameKey = 'pref_nickname';
static const _avatarIndexKey = 'pref_avatar_index';
static const _avatarImageKey = 'pref_avatar_image';
static const _profileBackgroundImageKey = 'pref_profile_background_image';
static const _aiApiBaseUrlKey = 'pref_ai_api_base_url';
static const _aiApiKeyKey = 'pref_ai_api_key';
static const _aiModelNameKey = 'pref_ai_model_name';
static const _legacyUserPrefsMigratedKey = 'pref_user_scoped_migrated';

Future<void> _load() async {
  await _loadForUser(_ref.read(authProvider).user?.id);
}

Future<void> handleAuthChanged(AuthState? previous, AuthState next) async {
  final previousUserId = previous?.user?.id;
  final nextUserId = next.user?.id;
  final signedOut =
    previous?.status != AuthStatus.unauthenticated &&
    next.status == AuthStatus.unauthenticated;

  if (!signedOut && previousUserId == nextUserId) return;

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 23

=====

(50 source lines on this page)

```

  await _loadForUser(nextUserId);
}

Future<void> _loadForUser(String? userId) async {
  state = const PreferencesState(isLoaded: false);

  if (userId == null || userId.isEmpty) {
    state = const PreferencesState(isLoaded: true);
    return;
  }

  await _migrateLegacyPreferencesIfNeeded(userId);

  final themeModeValue = await _storage.read(key: _scopedKey(userId, _themeMod
eKey));
  final nicknameValue = await _storage.read(key: _scopedKey(userId, _nicknameK
ey));
  final avatarValue = await _storage.read(key: _scopedKey(userId, _avatarIndex
Key));
  final avatarImageValue = await _storage.read(
    key: _scopedKey(userId, _avatarImageKey),
  );
  final profileBackgroundImageValue = await _storage.read(
    key: _scopedKey(userId, _profileBackgroundImageKey),
  );
  final aiApiBaseUrlValue = await _storage.read(
    key: _scopedKey(userId, _aiApiBaseUrlKey),
  );
  final aiApiKeyValue = await _storage.read(

```

```

        key: _scopedKey(userId, _aiApiKeyKey),
    );
    final aiModelNameValue = await _storage.read(
        key: _scopedKey(userId, _aiModelNameKey),
    );

    state = PreferencesState(
        themeMode: _decodeThemeMode(themeModeValue),
        nickname: nicknameValue ?? '',
        avatarIndex: int.tryParse(avatarValue ?? '') ?? 0,
        avatarImageData: avatarImageValue,
        profileBackgroundImageData: profileBackgroundImageValue,
        aiApiBaseUrl: aiApiBaseUrlValue ?? AiDefaults.baseUrl,
        aiApiKey: aiApiKeyValue ?? AiDefaults.apiKey,
        aiModelName: aiModelNameValue ?? AiDefaults.modelName,
        isLoading: true,
    );
}

Future<void> setThemeMode(ThemeMode themeMode) async {
    state = state.copyWith(themeMode: themeMode);
    await _writeScopedValue(_themeModeKey, _encodeThemeMode(themeMode));
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 24

=====

(50 source lines on this page)

```

Future<void> setNickname(String nickname) async {
    final trimmed = nickname.trim();
    state = state.copyWith(nickname: trimmed);
    await _writeScopedValue(_nicknameKey, trimmed);
}

Future<void> setAvatarIndex(int avatarIndex) async {
    state = state.copyWith(avatarIndex: avatarIndex, avatarImageData: null);
    await _writeScopedValue(_avatarIndexKey, '$avatarIndex');
    await _deleteScopedValue(_avatarImageKey);
}

Future<void> setAvatarImageData(String avatarImageData) async {
    state = state.copyWith(avatarImageData: avatarImageData);
    await _writeScopedValue(_avatarImageKey, avatarImageData);
}

Future<void> clearAvatarImageData() async {
    state = state.copyWith(avatarImageData: null);
    await _deleteScopedValue(_avatarImageKey);
}

Future<void> setProfileBackgroundImageData(
    String profileBackgroundImageData,
) async {
    state = state.copyWith(
        profileBackgroundImageData: profileBackgroundImageData,
    );
    await _writeScopedValue(
        _profileBackgroundImageKey,
        profileBackgroundImageData,
    );
}

```

```

    });
}

Future<void> clearProfileBackgroundImageData() async {
    state = state.copyWith(profileBackgroundImageData: null);
    await _deleteScopedValue(_profileBackgroundImageKey);
}

Future<void> setAiApiBaseUrl(String url) async {
    final trimmed = url.trim();
    state = state.copyWith(aiApiBaseUrl: trimmed);
    await _writeScopedValue(_aiApiBaseUrlKey, trimmed);
}

Future<void> setAiApiKey(String key) async {
    final trimmed = key.trim();
    state = state.copyWith(aiApiKey: trimmed);
    await _writeScopedValue(_aiApiKeyKey, trimmed);
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 25

=====

(50 source lines on this page)

```

Future<void> setAiModelName(String name) async {
    final trimmed = name.trim();
    state = state.copyWith(aiModelName: trimmed);
    await _writeScopedValue(_aiModelNameKey, trimmed);
}

Future<void> clearAiConfig() async {
    state = state.copyWith(
        aiApiBaseUrl: AiDefaults.baseUrl,
        aiApiKey: AiDefaults.apiKey,
        aiModelName: AiDefaults.modelName,
    );
    await _deleteScopedValue(_aiApiBaseUrlKey);
    await _deleteScopedValue(_aiApiKeyKey);
    await _deleteScopedValue(_aiModelNameKey);
}

String _scopedKey(String userId, String key) => '${key}_$userId';

String? _currentUserId() => _ref.read(authProvider).user?.id;

Future<void> _writeScopedValue(String key, String value) async {
    final userId = _currentUserId();
    if (userId == null || userId.isEmpty) return;
    await _storage.write(key: _scopedKey(userId, key), value: value);
}

Future<void> _deleteScopedValue(String key) async {
    final userId = _currentUserId();
    if (userId == null || userId.isEmpty) return;
    await _storage.delete(key: _scopedKey(userId, key));
}

Future<void> _migrateLegacyPreferencesIfNeeded(String userId) async {
    final migrated = await _storage.read(key: _legacyUserPrefsMigratedKey);
}

```

```

if (migrated == 'true') return;

final existingScopedValues = await Future.wait([
  _storage.read(key: _scopedKey(userId, _themeModeKey)),
  _storage.read(key: _scopedKey(userId, _nicknameKey)),
  _storage.read(key: _scopedKey(userId, _avatarIndexKey)),
  _storage.read(key: _scopedKey(userId, _avatarImageKey)),
  _storage.read(key: _scopedKey(userId, _profileBackgroundImageKey)),
]);
final hasScopedPrefs = existingScopedValues.any(
  (value) => value != null && value.isNotEmpty,
);
if (hasScopedPrefs) {
  await _storage.write(key: _legacyUserPrefsMigratedKey, value: 'true');
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 26

=====

(50 source lines on this page)

```

    return;
  }

  final themeModeValue = await _storage.read(key: _themeModeKey);
  final nicknameValue = await _storage.read(key: _nicknameKey);
  final avatarValue = await _storage.read(key: _avatarIndexKey);
  final avatarImageValue = await _storage.read(key: _avatarImageKey);
  final profileBackgroundImageValue = await _storage.read(
    key: _profileBackgroundImageKey,
  );

  final hasLegacyUserPrefs =
    (themeModeValue?.isNotEmpty ?? false) ||
    (nicknameValue?.isNotEmpty ?? false) ||
    (avatarValue?.isNotEmpty ?? false) ||
    (avatarImageValue?.isNotEmpty ?? false) ||
    (profileBackgroundImageValue?.isNotEmpty ?? false);

  if (!hasLegacyUserPrefs) {
    await _storage.write(key: _legacyUserPrefsMigratedKey, value: 'true');
    return;
  }

  if (themeModeValue != null && themeModeValue.isNotEmpty) {
    await _storage.write(
      key: _scopedKey(userId, _themeModeKey),
      value: themeModeValue,
    );
  }
  await _storage.write(
    key: _scopedKey(userId, _nicknameKey),
    value: nicknameValue ?? '',
  );
  await _storage.write(
    key: _scopedKey(userId, _avatarIndexKey),
    value: avatarValue ?? '0',
  );
  if (avatarImageValue != null && avatarImageValue.isNotEmpty) {
    await _storage.write(
      key: _scopedKey(userId, _avatarImageKey),
      value: avatarImageValue,
    );
  }

```

```

    );
  }
  if (profileBackgroundImageValue != null &&
      profileBackgroundImageValue.isNotEmpty) {
    await _storage.write(
      key: _scopedKey(userId, _profileBackgroundImageKey),
      value: profileBackgroundImageValue,
    );
  }
}

```

-----

Liqing Accounting Software V1.0      FIRST PAGE 27

=====

(50 source lines on this page)

```

    await _storage.delete(key: _themeModeKey);
    await _storage.delete(key: _nicknameKey);
    await _storage.delete(key: _avatarIndexKey);
    await _storage.delete(key: _avatarImageKey);
    await _storage.delete(key: _profileBackgroundImageKey);
    await _storage.write(key: _legacyUserPrefsMigratedKey, value: 'true');
  }

String _encodeThemeMode(ThemeMode themeMode) {
  switch (themeMode) {
    case ThemeMode.light:
      return 'light';
    case ThemeMode.dark:
      return 'dark';
    case ThemeMode.system:
      return 'system';
  }
}

ThemeMode _decodeThemeMode(String? value) {
  switch (value) {
    case 'light':
      return ThemeMode.light;
    case 'dark':
      return ThemeMode.dark;
    case 'system':
    default:
      return ThemeMode.system; // -----
  }
}

final preferencesProvider =
  StateNotifierProvider<PreferencesNotifier, PreferencesState>((ref) {
    final notifier = PreferencesNotifier(ref);
    ref.listen<AuthState>(authProvider, (previous, next) {
      notifier.handleAuthChanged(previous, next);
    });
    return notifier;
  });

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../providers/auth_provider.dart';
import '../features/auth/login_page.dart';
import '../features/auth/privacy_policy_page.dart';

```

```
import '../features/auth/terms_of_service_page.dart';
import '../features/home/home_page.dart';
import '../features/stats/stats_page.dart';
import '../features/bills/bills_page.dart';
```

-----

Liqing Accounting Software V1.0      FIRST PAGE 28

=====

(50 source lines on this page)

```
import '../features/profile/profile_page.dart';
import '../features/profile/about_page.dart';
import '../shared/widgets/main_shell.dart';

final routerProvider = Provider<GoRouter>((ref) {
  final authStatus = ref.watch(authProvider.select((state) => state.status));

  return GoRouter(
    redirect: (context, state) {
      final isAuth = authStatus == AuthStatus.authenticated;
      final isAuthRoute = state.matchedLocation.startsWith('/auth');
      final isRootRoute = state.matchedLocation == '/';
      final isLegalRoute =
        state.matchedLocation == '/legal/privacy' ||
        state.matchedLocation == '/legal/terms' ||
        state.matchedLocation == '/about';

      if (isRootRoute) return '/home';
      if (authStatus == AuthStatus.unknown) return null;
      if (!isAuth && !isAuthRoute && !isLegalRoute) return '/auth/login';
      if (isAuth && isAuthRoute) return '/home';
      return null;
    },
    routes: [
      ShellRoute(
        builder: (context, state, child) => MainShell(child: child),
        routes: [
          GoRoute(
            path: '/home',
            pageBuilder: (context, state) =>
              const NoTransitionPage(child: HomePage()),
          ),
          GoRoute(
            path: '/stats',
            pageBuilder: (context, state) =>
              const NoTransitionPage(child: StatsPage()),
          ),
          GoRoute(
            path: '/bills',
            pageBuilder: (context, state) =>
              const NoTransitionPage(child: BillsPage()),
          ),
          GoRoute(
            path: '/profile',
            pageBuilder: (context, state) =>
              const NoTransitionPage(child: ProfilePage()),
          ),
        ],
      ),
    ],
  ),
  GoRoute(
```

=====

(50 source lines on this page)

```

    path: '/auth/login',
    builder: (context, state) => const LoginPage(),
  ),
  GoRoute(
    path: '/legal/privacy',
    builder: (context, state) => const PrivacyPolicyPage(),
  ),
  GoRoute(
    path: '/legal/terms',
    builder: (context, state) => const TermsOfServicePage(),
  ),
  GoRoute(path: '/about', builder: (context, state) => const AboutPage()),
],
);
});
String aiChatCompletionsUrl(String input) {
  final base = _normalized(input);
  if (base.endsWith('/chat/completions')) return base;
  if (base.endsWith('/v1')) return '$base/chat/completions';
  return '$base/v1/chat/completions';
}

String aiModelsUrl(String input) {
  final base = _normalized(input);
  if (base.endsWith('/models')) return base;
  if (base.endsWith('/v1')) return '$base/models';
  return '$base/v1/models';
}

String aiAudioTranscriptionsUrl(String input) {
  final base = _normalized(input);
  if (base.endsWith('/audio/transcriptions')) return base;
  if (base.endsWith('/v1')) return '$base/audio/transcriptions';
  return '$base/v1/audio/transcriptions';
}

String _normalized(String input) => input.trim().replaceAll(RegExp(r'/+$'), '');
import 'dart:convert';

import 'package:dio/dio.dart';

import 'ai_endpoint_utils.dart';

/// AI --- - --- OpenAI --- API -----
class AiService {
  final String baseUrl;
  final String apiKey;
  final String model;

  AiService({

```

=====

(50 source lines on this page)

```

    required this.baseUrl,
    required this.apiKey,
    this.model = 'deepseek-v4-flash',
  });

  /// ----- AI ---
  Stream<String> chatStream({
    required List<Map<String, String>> messages,
  }) async* {
    final url = aiChatCompletionsUrl(baseUrl);
    final dio = Dio();
    dio.options.headers = {
      'Authorization': 'Bearer $apiKey',
      'Content-Type': 'application/json',
    };
    dio.options.sendTimeout = const Duration(seconds: 30);
    dio.options.receiveTimeout = const Duration(minutes: 5);

    final body = jsonEncode({
      'model': model,
      'messages': messages,
      'stream': true,
      'temperature': 0.3,
    });

    try {
      final response = await dio.post<ResponseBody>(
        url,
        data: body,
        options: Options(responseType: ResponseType.stream),
      );

      final stream = response.data?.stream;
      if (stream == null) {
        yield* Stream.error('-----');
        return;
      }

      final buffer = StringBuffer();
      await for (final chunk in stream) {
        final text = utf8.decode(chunk, allowMalformed: true);
        buffer.write(text);

        // ----- SSE ---
        final lines = buffer.toString().split('\n');
        buffer.clear();

        for (var i = 0; i < lines.length - 1; i++) {
          final line = lines[i].trim();
          if (!line.startsWith('data: ')) continue;

```

```

await app.register(billsRoutes);
await app.register(categoryRoutes);
await app.register(transactionRoutes);
await app.register(exportRoutes);

```

```

// -----
app.get("/api/health", async () => ({ status: "ok" }));

return app;
}
import type { FastifyInstance } from "fastify";
import fp from "fastify-plugin";
import { getPrisma } from "../utils/prisma.js";

declare module "fastify" {
  interface FastifyRequest {
    userId: string;
  }
}

export default fp(async function authPlugin(app: FastifyInstance) {
  app.decorateRequest("userId", "");

  app.addHook("onRequest", async (request, reply) => {
    // -----
    const publicRoutes = ["/api/auth/login", "/api/auth/register", "/api/auth/se
nd-code", "/api/auth/verify-code", "/api/health", "/docs"];
    const isPublic = publicRoutes.some((r) => request.url.startsWith(r));
    if (isPublic) return;

    try {
      const decoded = await request.jwtVerify<{ sub: string }>();
      request.userId = decoded.sub;
    } catch {
      reply.status(401).send({ error: "___" });
    }
  });
});

import type { FastifyInstance } from "fastify";
import fp from "fastify-plugin";
import cors from "@fastify/cors";
import helmet from "@fastify/helmet";
import rateLimit from "@fastify/rate-limit";
import jwt from "@fastify/jwt";
import swagger from "@fastify/swagger";
import swaggerUI from "@fastify/swagger-ui";

export default fp(async function corePlugins(app: FastifyInstance) {
  const isDev = process.env.NODE_ENV !== "production";

  // ____
  -----

```

Liqing Accounting Software V1.0      LAST PAGE 02

=====

(50 source lines on this page)

```

await app.register(helmet);

// ____ IP ____ 10
await app.register(rateLimit, {
  max: 100,
  timeWindow: "1 minute",
  keyGenerator: (request) => {
    return request.ip;
  }
});

```

```

    },
  });

  // CORS
  await app.register(cors, {
    origin: isDev
      ? true
      : (
          process.env.CORS_ORIGIN ||
          "https://liqing.eatdesk.net"
        ).split(", "),
  });

  // JWT
  if (!process.env.JWT_SECRET) {
    throw new Error("JWT_SECRET environment variable is required in production");
  }
  await app.register(jwt, {
    secret: process.env.JWT_SECRET,
  });

  // Swagger
  if (isDev) {
    await app.register(swagger, {
      openapi: {
        info: {
          title: "OneKeep API",
          version: "1.0.0",
          description: "OneKeep API",
        },
      },
    });

    await app.register(swaggerUI, {
      routePrefix: "/docs",
    });
  }
});

import type { FastifyInstance } from "fastify";
import bcrypt from "bcryptjs";
import crypto from "node:crypto";
import { getPrisma } from "../utils/prisma.js";

```

-----

Liqing Accounting Software V1.0      LAST PAGE 03

=====

(50 source lines on this page)

```

import { registerSchema, loginSchema, meSchema, sendCodeSchema, verifyCodeSchema }
  from "../schemas/auth.js";
import { ensureUserCategories } from "../utils/default-categories.js";

function serializeUser(user: {
  id: string;
  username: string | null;
  email: string;
  name: string;
}) {
  return {
    id: user.id,

```

```

        username: user.username,
        email: user.email,
        name: user.name,
        displayName: user.name,
    }
};

async function findOrCreateEmailUser(prisma: ReturnType<typeof getPrisma>, email
: string) {
    let user = await prisma.user.findUnique({ where: { email } });
    let isNewUser = false;

    if (!user) {
        const defaultName = email.split("@")[0];
        const randomSuffix = crypto.randomBytes(3).toString("hex");
        user = await prisma.user.create({
            data: {
                email,
                username: `user_${randomSuffix}`,
                password: "",
                name: defaultName,
            },
        });
        await ensureUserCategories(prisma, user.id);
        isNewUser = true;
    }

    return { user, isNewUser };
}

export default async function authRoutes(app: FastifyInstance) {
    const prisma = getPrisma();

    app.get("/api/auth/me", { schema: meSchema }, async (request) => {
        const user = await prisma.user.findUniqueOrThrow({
            where: { id: request.userId },
            select: {
                id: true,
                username: true,
                email: true,
            },
        });
    });
}

```

-----

Liqing Accounting Software V1.0      LAST PAGE 04

=====

(50 source lines on this page)

```

        name: true,
    },
});

return {
    user: serializeUser(user),
};
});

app.post("/api/auth/register", { schema: registerSchema }, async (request, rep
ly) => {
    const { username, email, displayName, password, code } = request.body as {
        username: string;
        email: string;
        displayName: string;
    };
}

```

```

    password: string;
    code?: string;
};

const normalizedUsername = username.trim().toLowerCase();
const normalizedEmail = email.trim().toLowerCase();
const normalizedDisplayName = displayName.trim();

const existingUser = await prisma.user.findFirst({
  where: {
    OR: [{ email: normalizedEmail }, { username: normalizedUsername }],
  },
});

if (existingUser?.email === normalizedEmail) {
  return reply.status(409).send({ error: "-----" });
}
if (existingUser?.username === normalizedUsername) {
  return reply.status(409).send({ error: "-----" });
}

if (code) {
  const record = await prisma.verificationCode.findFirst({
    where: {
      email: normalizedEmail,
      code,
      used: false,
      expiresAt: { gte: new Date() },
    },
    orderBy: { createdAt: "desc" },
  });

  if (!record) {
    return reply.status(400).send({ error: "-----" });
  }
}

```

-----

Liqing Accounting Software V1.0      LAST PAGE 05

=====

(50 source lines on this page)

```

    await prisma.verificationCode.update({
      where: { id: record.id },
      data: { used: true },
    });
  }

  const hashed = await bcrypt.hash(password, 10);
  const user = await prisma.user.create({
    data: {
      username: normalizedUsername,
      email: normalizedEmail,
      password: hashed,
      name: normalizedDisplayName,
    },
  });
  await ensureUserCategories(prisma, user.id);

  const token = app.jwt.sign({ sub: user.id }, { expiresIn: "7d" });
  return {

```

```

        token,
        user: serializeUser(user),
    });
});

app.delete("/api/auth/account", async (request, reply) => {
    const userId = request.userId;

    // -----
    await prisma.transaction.deleteMany({ where: { userId } });
    await prisma.category.deleteMany({ where: { userId } });
    // -----
    await prisma.user.delete({ where: { id: userId } });

    return { success: true };
});

app.post("/api/auth/login", { schema: loginSchema }, async (request, reply) =>
{
    const { identifier, password } = request.body as {
        identifier: string;
        password: string;
    };

    const normalizedIdentifier = identifier.trim().toLowerCase();
    const user = await prisma.user.findFirst({
        where: {
            OR: [{ email: normalizedIdentifier }, { username: normalizedIdentifier }
],
        },
    });

    if (!user) {

```

-----

Liqing Accounting Software V1.0      LAST PAGE 06

=====

(50 source lines on this page)

```

    return reply.status(401).send({ error: "___/_____" });
}

const valid = await bcrypt.compare(password, user.password);
if (!valid) {
    return reply.status(401).send({ error: "___/_____" });
}

const token = app.jwt.sign({ sub: user.id }, { expiresIn: "7d" });
return {
    token,
    user: serializeUser(user),
};
});

// ===== _/_ _=====

app.post("/api/auth/send-code", { schema: sendCodeSchema }, async (request, re
ply) => {
    const { email } = request.body as { email: string };
    const normalizedEmail = email.trim().toLowerCase();

```

```

// __: ____ 60 _____
const recent = await prisma.verificationCode.findFirst({
  where: {
    email: normalizedEmail,
    createdAt: { gte: new Date(Date.now() - 60 * 1000) },
  },
  orderBy: { createdAt: "desc" },
});
if (recent) {
  const wait = Math.ceil((recent.createdAt.getTime() + 60000 - Date.now()) /
1000);
  return reply.status(429).send({ error: `_ ${wait} ____` });
}

// __ 6 ____
const code = String(Math.floor(100000 + Math.random() * 900000));
const expiresAt = new Date(Date.now() + 10 * 60 * 1000); // 10 ____

await prisma.verificationCode.create({
  data: { email: normalizedEmail, code, expiresAt },
});

// _____ Resend _____
const resendKey = process.env.RESEND_API_KEY;
if (resendKey) {
  try {
    const { Resend } = await import("resend");
    const resend = new Resend(resendKey);
    const emailPayload = {
      from: process.env.RESEND_FROM || "__ <noreply@eatdesk.net>",

```

-----

Liqing Accounting Software V1.0      LAST PAGE 07

=====

(50 source lines on this page)

```

    to: normalizedEmail,
    subject: "_____"
    html: `<div style="font-family: sans-serif; padding: 24px;">
      <h2 style="color: #10B981;">__</h2>
      <p>____</p>
      <div style="font-size: 32px; letter-spacing: 8px; font-weight: bold;
color: #10B981; padding: 16px; text-align: center;">
        ${code}
      </div>
      <p style="color: #666;">__ 10 _____</p>
    </div>`,
  };

let lastError: unknown;
for (let attempt = 1; attempt <= 2; attempt += 1) {
  const result = await resend.emails.send(emailPayload);
  if (!result.error) {
    lastError = undefined;
    break;
  }
  lastError = result.error;
  request.log.warn({ attempt, error: result.error }, "Failed to send ema
il via Resend");
  if (attempt < 2) {
    await new Promise((resolve) => setTimeout(resolve, 800));

```

```

    }
  }
  if (lastError) {
    return reply.status(502).send({ error: "-----" });
  }
} catch (e) {
  request.log.warn(e, "Failed to send email via Resend");
  return reply.status(502).send({ error: "-----" });
}
} else if (process.env.NODE_ENV !== "production") {
  request.log.info(`[DEV] Verification code for ${normalizedEmail}: ${code}`
);
} else {
  request.log.error("RESEND_API_KEY is not configured");
  return reply.status(503).send({ error: "-----" });
}

return { success: true };
});

app.post("/api/auth/verify-code", { schema: verifyCodeSchema }, async (request
, reply) => {
  const { email, code } = request.body as { email: string; code: string };
  const normalizedEmail = email.trim().toLowerCase();
  const reviewEmail = process.env.REVIEW_ACCOUNT_EMAIL?.trim().toLowerCase();
  const reviewCode = process.env.REVIEW_ACCOUNT_CODE?.trim();

  if (reviewEmail && reviewCode && normalizedEmail === reviewEmail && code ===
reviewCode) {

```

-----

Liqing Accounting Software V1.0      LAST PAGE 08

=====

(50 source lines on this page)

```

  const { user, isNewUser } = await findOrCreateEmailUser(prisma, normalized
Email);
  const token = app.jwt.sign({ sub: user.id }, { expiresIn: "7d" });
  return {
    token,
    user: serializeUser(user),
    isNewUser,
  };
}

// -----
const record = await prisma.verificationCode.findFirst({
  where: {
    email: normalizedEmail,
    code,
    used: false,
    expiresAt: { gte: new Date() },
  },
  orderBy: { createdAt: "desc" },
});

if (!record) {
  return reply.status(400).send({ error: "-----" });
}

```

```

// -----
await prisma.verificatiOnCode.update({
  where: { id: record.id },
  data: { used: true },
});

// -----
const { user, isNewUser } = await findOrCreateEmailUser(prisma, normalizedEmail);

const token = app.jwt.sign({ sub: user.id }, { expiresIn: "7d" });
return {
  token,
  user: serializeUser(user),
  isNewUser,
};
});
}

import type { FastifyInstance } from "fastify";
import { getPrisma } from "../utils/prisma.js";
import { monthRange, toNumber } from "../utils/date.js";
import { billsListSchema } from "../schemas/bills.js";
import type { Prisma } from "@prisma/client";

export default async function billsRoutes(app: FastifyInstance) {
  const prisma = getPrisma();

```

-----

Liqing Accounting Software V1.0      LAST PAGE 09

=====

(50 source lines on this page)

```

// GET /api/bills
app.get("/api/bills", { schema: billsListSchema }, async (request) => {
  const { month, filterType, query, cursor, pageSize } = request.query as {
    month?: string;
    filterType?: string;
    query?: string;
    cursor?: string;
    pageSize?: number;
  };

  const userId = request.userId;
  const take = pageSize ?? 20;

  const where: Prisma.TransactionWhereInput = { userId };

  // ----
  if (month) {
    const { start, end } = monthRange(month);
    where.occurredAt = { gte: start, lt: end };
  }

  // -----
  if (filterType && filterType !== "all") {
    where.direction = filterType as "expense" | "income";
  }

  // --
  if (query) {
    where.OR = [

```

```

    { title: { contains: query } },
    { note: { contains: query } },
    { category: { name: { contains: query } } } },
  ];
}

// -----
const findArgs: Prisma.TransactionFindManyArgs = {
  where,
  orderBy: { occurredAt: "desc" },
  take: take + 1, // -----
  include: { category: { select: { name: true, icon: true } } },
};
if (cursor) {
  findArgs.cursor = { id: cursor };
  findArgs.skip = 1;
}

const items = await prisma.transaction.findMany(findArgs);
const hasMore = items.length > take;
if (hasMore) items.pop();

```

-----

Liqing Accounting Software V1.0      LAST PAGE 10

=====

(50 source lines on this page)

```

// -----
const groups = new Map<
  string,
  {
    date: string;
    items: typeof items;
    expenseTotal: number;
    incomeTotal: number;
  }
>();

for (const item of items) {
  const dateKey = item.occurredAt.toISOString().slice(0, 10);
  let group = groups.get(dateKey);
  if (!group) {
    group = { date: dateKey, items: [], expenseTotal: 0, incomeTotal: 0 };
    groups.set(dateKey, group);
  }
  group.items.push(item);
  const amt = toNumber(item.amount);
  if (item.direction === "expense") group.expenseTotal += amt;
  else group.incomeTotal += amt;
}

const groupedBills = Array.from(groups.values()).map((g) => ({
  date: g.date,
  summary: {
    expense: g.expenseTotal,
    income: g.incomeTotal,
  },
  items: g.items.map((t) => {
    const cat = (t as any).category as { name: string; icon: string };
    return {

```

```

        transactionId: t.id,
        categoryId: t.categoryId,
        title: t.title,
        categoryName: cat.name,
        categoryIcon: cat.icon,
        occurredAt: t.occurredAt.toISOString(),
        amount: toNumber(t.amount),
        direction: t.direction,
        note: t.note,
        merchant: t.merchant,
    });
    });
    });
    return {
        groupedBills,
        nextCursor: hasMore ? items[items.length - 1]?.id : null,
    };
}

```

-----

Liqing Accounting Software V1.0      LAST PAGE 11

=====

(50 source lines on this page)

```

    });
    });
}
import type { FastifyInstance, FastifyReply } from "fastify";
import type { TransactionType } from "@prisma/client";
import { getPrisma } from "../utils/prisma.js";
import {
    createCategorySchema,
    deleteCategorySchema,
    listCategoriesSchema,
    reorderCategoriesSchema,
    updateCategorySchema,
} from "../schemas/category.js";
import {
    ensureUserCategories,
    resequenceUserCategories,
} from "../utils/default-categories.js";

function serializeCategory(category: {
    id: string;
    name: string;
    icon: string;
    type: TransactionType;
    sort: number;
}) {
    return {
        id: category.id,
        name: category.name,
        icon: category.icon,
        type: category.type,
        sort: category.sort,
    };
}

async function findOwnedCategory(
    prisma: ReturnType<typeof getPrisma>,
    userId: string,
    categoryId: string,

```

```

    reply: FastifyReply,
  ) {
    const category = await prisma.category.findFirst({
      where: { id: categoryId, userId },
    });

    if (!category) {
      reply.status(404).send({ error: "-----" });
      return null;
    }

    return category;
  }
}

```

-----

Liqing Accounting Software V1.0      LAST PAGE 12

=====

(50 source lines on this page)

```

}

export default async function categoryRoutes(app: FastifyInstance) {
  const prisma = getPrisma();

  app.get("/api/categories", { schema: listCategoriesSchema }, async (request) => {
    await ensureUserCategories(prisma, request.userId);

    const categories = await prisma.category.findMany({
      where: { userId: request.userId },
      orderBy: [{ type: "asc" }, { sort: "asc" }, { id: "asc" }],
    });

    return { categories: categories.map(serializeCategory) };
  });

  app.post("/api/categories", { schema: createCategorySchema }, async (request,
  reply) => {
    const { name, icon, type } = request.body as {
      name: string;
      icon: string;
      type: TransactionType;
    };
    const userId = request.userId;

    await ensureUserCategories(prisma, userId);

    const sort = await prisma.category.count({
      where: { userId, type },
    });

    const category = await prisma.category.create({
      data: {
        name: name.trim(),
        icon,
        type,
        sort: sort + 1,
        userId,
      },
    });

    return reply.status(201).send({ category: serializeCategory(category) });
  }
}

```

```

});

app.put("/api/categories/reorder", { schema: reorderCategoriesSchema }, async
(request, reply) => {
  const { type, categoryIds } = request.body as {
    type: TransactionType;
    categoryIds: string[];
  };
  const userId = request.userId;

```

-----

Li Qing Accounting Software V1.0      LAST PAGE 13

=====

(50 source lines on this page)

```

  const categories = await prisma.category.findMany({
    where: { userId, type },
    select: { id: true },
    orderBy: [{ sort: "asc" }, { id: "asc" }],
  });
  const ownedIds = categories.map((category) => category.id);

  if (
    ownedIds.length !== categoryIds.length ||
    ownedIds.some((id) => !categoryIds.includes(id))
  ) {
    return reply.status(400).send({ error: "-----" });
  }

  await prisma.$transaction(
    categoryIds.map((id, index) =>
      prisma.category.update({
        where: { id },
        data: { sort: index + 1 },
      })
    ),
  );

  const updated = await prisma.category.findMany({
    where: { userId },
    orderBy: [{ type: "asc" }, { sort: "asc" }, { id: "asc" }],
  });

  return { categories: updated.map(serializeCategory) };
});

app.put("/api/categories/:id", { schema: updateCategorySchema }, async (request,
reply) => {
  const { id } = request.params as { id: string };
  const { name, icon, type } = request.body as {
    name?: string;
    icon?: string;
    type?: TransactionType;
  };
  const userId = request.userId;

  const category = await findOwnedCategory(prisma, userId, id, reply);
  if (!category) return;

  const nextType = type ?? category.type;

```

```

if (type && type !== category.type) {
  const txCount = await prisma.transaction.count({
    where: { categoryId: category.id },
  });
  if (txCount > 0) {
    return reply.status(400).send({ error: "_____/_____" });
  }
}

```

-----

Li Qing Accounting Software V1.0      LAST PAGE 14

=====

(50 source lines on this page)

```

    }
  }

  const nextSort =
    nextType === category.type
      ? category.sort
      : (await prisma.category.count({ where: { userId, type: nextType } }))) +
1;

  const updated = await prisma.category.update({
    where: { id: category.id },
    data: {
      name: name?.trim() ?? category.name,
      icon: icon ?? category.icon,
      type: nextType,
      sort: nextSort,
    },
  });

  if (nextType !== category.type) {
    await resequenceUserCategories(prisma, userId, category.type);
    await resequenceUserCategories(prisma, userId, nextType);
  }

  return { category: serializeCategory(updated) };
});

app.delete("/api/categories/:id", { schema: deleteCategorySchema }, async (req
uest, reply) => {
  const { id } = request.params as { id: string };
  const userId = request.userId;

  const category = await findOwnedCategory(prisma, userId, id, reply);
  if (!category) return;

  const txCount = await prisma.transaction.count({
    where: { categoryId: category.id },
  });
  if (txCount > 0) {
    return reply.status(400).send({ error: "_____" });
  }

  await prisma.category.delete({
    where: { id: category.id },
  });
  await resequenceUserCategories(prisma, userId, category.type);

  return reply.status(204).send();
});

```

```

}
import type { FastifyInstance } from "fastify";
import { getPrisma } from "../utils/prisma.js";

```

-----

Liqing Accounting Software V1.0      LAST PAGE 15

=====

(50 source lines on this page)

```

import { toNumber } from "../utils/date.js";
import { exportDataSchema } from "../schemas/export.js";
import ExcelJS from "exceljs";

```

```

interface ExportQuery {
  startMonth?: string;
  endMonth?: string;
}

```

```

interface TransactionRow {
  occurredAt: string;
  direction: string;
  categoryName: string;
  title: string;
  amount: number;
  merchant: string;
  note: string;
}

```

```

function buildDateRange(startMonth?: string, endMonth?: string) {
  const now = new Date();
  const start = startMonth
    ? new Date(`${startMonth}-01T00:00:00.000Z`)
    : new Date(Date.UTC(now.getFullYear(), now.getMonth(), 1));

  let end: Date;
  if (endMonth) {
    const [y, m] = endMonth.split("-").map(Number);
    end = new Date(Date.UTC(y, m, 1)); // first day of next month
  } else {
    end = new Date(Date.UTC(now.getFullYear(), now.getMonth() + 1, 1));
  }

  return { start, end };
}

```

```

async function fetchTransactions(userId: string, startMonth?: string, endMonth?:
string): Promise<TransactionRow[]> {
  const prisma = getPrisma();
  const { start, end } = buildDateRange(startMonth, endMonth);

  const items = await prisma.transaction.findMany({
    where: {
      userId,
      occurredAt: { gte: start, lt: end },
    },
    orderBy: { occurredAt: "desc" },
    include: {
      category: { select: { name: true } },
    },
  });
}

```

=====

(50 source lines on this page)

```

    return items.map((t) => ({
      occurredAt: t.occurredAt.toISOString().slice(0, 10),
      direction: t.direction === "expense" ? "支出" : "收入",
      categoryName: (t as any).category?.name ?? "___",
      title: t.title,
      amount: toNumber(t.amount),
      merchant: t.merchant ?? "",
      note: t.note ?? "",
    }));
  }

const CSV_HEADERS = ["__", "__", "__", "__", "__", "__", "__"];

function buildCsv(rows: TransactionRow[]): string {
  // BOM for Excel to recognize UTF-8
  const bom = "\uFEFF";
  const lines: string[] = [CSV_HEADERS.join(",")];

  for (const r of rows) {
    const cells = [
      r.occurredAt,
      r.direction,
      escapeCsvField(r.categoryName),
      escapeCsvField(r.title),
      r.amount.toFixed(2),
      escapeCsvField(r.merchant),
      escapeCsvField(r.note),
    ];
    lines.push(cells.join(","));
  }

  return bom + lines.join("\n");
}

function escapeCsvField(value: string): string {
  if (value.includes(",") || value.includes('"') || value.includes("\n")) {
    return `"${value.replace(/"/g, '"')}"`;
  }
  return value;
}

async function buildXlsx(rows: TransactionRow[]): Promise<Buffer> {
  const workbook = new ExcelJS.Workbook();
  workbook.creator = "OneKeep";
  workbook.created = new Date();

  const sheet = workbook.addWorksheet("____");

  // Define columns

```

=====

(50 source lines on this page)

```

sheet.columns = [
  { header: "--", key: "occurredAt", width: 14 },
  { header: "--", key: "direction", width: 8 },
  { header: "--", key: "categoryName", width: 12 },
  { header: "--", key: "title", width: 20 },
  { header: "--", key: "amount", width: 12 },
  { header: "--", key: "merchant", width: 18 },
  { header: "--", key: "note", width: 24 },
];

// Style header row
const headerRow = sheet.getRow(1);
headerRow.font = { bold: true, color: { argb: "FFFFFF" } };
headerRow.fill = {
  type: "pattern",
  pattern: "solid",
  fgColor: { argb: "FF10B981" },
};
headerRow.alignment = { vertical: "middle", horizontal: "center" };
headerRow.height = 28;

// Add data rows
for (const r of rows) {
  const row = sheet.addRow({
    occurredAt: r.occurredAt,
    direction: r.direction,
    categoryName: r.categoryName,
    title: r.title,
    amount: r.amount,
    merchant: r.merchant,
    note: r.note,
  });

  // Color-code expense/income
  const directionCell = row.getCell("direction");
  if (r.direction === "--") {
    directionCell.font = { color: { argb: "FFFF6B6B" } };
  } else {
    directionCell.font = { color: { argb: "FF10B981" } };
  }

  // Number format for amount
  row.getCell("amount").numFmt = "#,##0.00";
}

// Freeze header row
sheet.views = [{ state: "frozen", ySplit: 1 }];

const buffer = await workbook.xlsx.writeBuffer();
return Buffer.from(buffer);

```

-----

Liqing Accounting Software V1.0      LAST PAGE 18

=====

(50 source lines on this page)

```

}

export default async function exportRoutes(app: FastifyInstance) {
  // GET /api/export/csv

```

```

    app.get("/api/export/csv", { schema: exportDataSchema }, async (request, reply) => {
      const userId = request.userId;
      const { startMonth, endMonth } = request.query as ExportQuery;

      const rows = await fetchTransactions(userId, startMonth, endMonth);
      const csv = buildCsv(rows);

      const filename = `onekeep-export-${startMonth ?? "all"}-${endMonth ?? "all"}.csv`;

      reply
        .header("Content-Type", "text/csv; charset=utf-8")
        .header("Content-Disposition", `attachment; filename="${filename}"`)
        .send(csv);
    });

    // GET /api/export/xlsx
    app.get("/api/export/xlsx", { schema: exportDataSchema }, async (request, reply) => {
      const userId = request.userId;
      const { startMonth, endMonth } = request.query as ExportQuery;

      const rows = await fetchTransactions(userId, startMonth, endMonth);
      const buffer = await buildXlsx(rows);

      const filename = `onekeep-export-${startMonth ?? "all"}-${endMonth ?? "all"}.xlsx`;

      reply
        .header("Content-Type", "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
        .header("Content-Disposition", `attachment; filename="${filename}"`)
        .send(buffer);
    });
  }

  import type { FastifyInstance } from "fastify";
  import { getPrisma } from "../utils/prisma.js";
  import { monthRange, toNumber } from "../utils/date.js";
  import { homeSummarySchema } from "../schemas/home.js";

  export default async function homeRoutes(app: FastifyInstance) {
    const prisma = getPrisma();

    // GET /api/home/summary?month=2026-03
    app.get("/api/home/summary", { schema: homeSummarySchema }, async (request) => {
      const { month } = request.query as { month: string };
      const userId = request.userId;
      const { start, end } = monthRange(month);

      // ----

```

```

    const [user, aggregates, recentTransactions] = await Promise.all([
      prisma.user.findUniqueOrThrow({
        where: { id: userId },

```

```

    select: { id: true, name: true, email: true },
  }},

  prisma.transaction.groupBy({
    by: ["direction"],
    where: { userId, occurredAt: { gte: start, lt: end } },
    _sum: { amount: true },
  }),

  prisma.transaction.findMany({
    where: { userId, occurredAt: { gte: start, lt: end } },
    orderBy: { occurredAt: "desc" },
    take: 10,
    include: { category: { select: { name: true, icon: true } } },
  })),
]);

const income = aggregates.find((a) => a.direction === "income");
const expense = aggregates.find((a) => a.direction === "expense");

const totalIncome = income?._sum.amount ? toNumber(income._sum.amount) : 0;
const totalExpense = expense?._sum.amount ? toNumber(expense._sum.amount) :
0;

return {
  user,
  balanceSummary: {
    amount: totalIncome - totalExpense,
  },
  incomeSummary: { amount: totalIncome },
  expenseSummary: { amount: totalExpense },
  recentTransactions: recentTransactions.map((t) => ({
    transactionId: t.id,
    categoryId: t.categoryId,
    title: t.title,
    categoryName: t.category.name,
    categoryIcon: t.category.icon,
    occurredAt: t.occurredAt.toISOString(),
    amount: toNumber(t.amount),
    direction: t.direction,
    note: t.note,
    merchant: t.merchant,
  }))),
};
});
}
import type { FastifyInstance } from "fastify";
import { getPrisma } from "../utils/prisma.js";

-----

Liqing Accounting Software V1.0    LAST PAGE 20
=====
(50 source lines on this page)

import { monthRange, toNumber } from "../utils/date.js";
import { statsOverviewSchema } from "../schemas/stats.js";
import type { TransactionType } from "@prisma/client";

export default async function statsRoutes(app: FastifyInstance) {
  const prisma = getPrisma();

```

```

// GET /api/stats/overview?month=2026-03&metricType=expense
app.get("/api/stats/overview", { schema: statsOverviewSchema }, async (request
) => {
  const { month, metricType } = request.query as {
    month: string;
    metricType?: TransactionType;
  };
  const userId = request.userId;
  const { start, end } = monthRange(month);

  const whereBase = { userId, occurredAt: { gte: start, lt: end } };

  // --
  const aggregates = await prisma.transaction.groupBy({
    by: ["direction"],
    where: whereBase,
    _sum: { amount: true },
  });

  const incomeAgg = aggregates.find((a) => a.direction === "income");
  const expenseAgg = aggregates.find((a) => a.direction === "expense");

  const totalIncome = incomeAgg?._sum.amount ? toNumber(incomeAgg._sum.amount)
: 0;
  const totalExpense = expenseAgg?._sum.amount ? toNumber(expenseAgg._sum.amou
nt) : 0;

  // ----- Prisma ----- SQL ----- SQLite/MySQL
  const trendDirection = metricType ?? "expense";
  const trendTransactions = await prisma.transaction.findMany({
    where: { userId, direction: trendDirection, occurredAt: { gte: start, lt:
end } },
    select: { occurredAt: true, amount: true },
    orderBy: { occurredAt: "asc" },
  });

  // -----
  const dayMap = new Map<string, number>();
  for (const t of trendTransactions) {
    const day = t.occurredAt.toISOString().slice(0, 10);
    dayMap.set(day, (dayMap.get(day) ?? 0) + toNumber(t.amount));
  }
  const trendSeries = Array.from(dayMap.entries()).map(([day, total]) => ({
    label: day.slice(5), // "03-01"
    value: total,
  }));
}

```

-----

Liqing Accounting Software V1.0      LAST PAGE 21

=====

(50 source lines on this page)

```

// -----
const categoryRanks = await prisma.transaction.groupBy({
  by: ["categoryId"],
  where: { ...whereBase, direction: trendDirection },
  _sum: { amount: true },
  orderBy: { _sum: { amount: "desc" } },
  take: 10,
});

```

```

const categoryIds = categoryRanks.map((r) => r.categoryId);
const categories = await prisma.category.findMany({
  where: { id: { in: categoryIds } },
});
const catMap = new Map(categories.map((c) => [c.id, c]));

const maxAmount = categoryRanks[0]?._sum.amount
  ? toNumber(categoryRanks[0]._sum.amount)
  : 0;

return {
  totals: { income: totalIncome, expense: totalExpense },
  trendSeries,
  categoryRanks: categoryRanks.map((r) => {
    const cat = catMap.get(r.categoryId);
    const amount = r._sum.amount ? toNumber(r._sum.amount) : 0;
    return {
      categoryId: r.categoryId,
      categoryName: cat?.name ?? "",
      categoryIcon: cat?.icon ?? "",
      amount,
      progressBase: maxAmount,
      progressRatio: maxAmount > 0 ? amount / maxAmount : 0,
    };
  }),
};
});
});
});
import type { FastifyInstance } from "fastify";
import { getPrisma } from "../utils/prisma.js";
import {
  createTransactionSchema,
  deleteTransactionSchema,
  updateTransactionSchema,
} from "../schemas/transaction.js";

export default async function transactionRoutes(app: FastifyInstance) {
  const prisma = getPrisma();

  async function findAccessibleCategory(userId: string, categoryId: string) {
    return prisma.category.findFirst({

```

-----

Liqing Accounting Software V1.0      LAST PAGE 22

=====

(50 source lines on this page)

```

      where: {
        id: categoryId,
        OR: [{ userId }, { userId: null }],
      },
    });
  }

  app.post("/api/transactions", { schema: createTransactionSchema }, async (request, reply) => {
    const { title, amount, direction, categoryId, occurredAt, note, merchant } =
      request.body as {
        title: string;
        amount: number;

```

```

    direction: "expense" | "income";
    categoryId: string;
    occurredAt: string;
    note?: string;
    merchant?: string;
  };
  const userId = request.userId;

  const category = await findAccessibleCategory(userId, categoryId);
  if (!category) {
    return reply.status(400).send({ error: "_____" });
  }
  if (category.type !== direction) {
    return reply.status(400).send({ error: "_____" });
  }

  const tx = await prisma.transaction.create({
    data: {
      title,
      amount,
      direction,
      categoryId,
      occurredAt: new Date(occurredAt),
      note,
      merchant,
      userId,
    },
  });

  return reply.status(201).send({
    transactionId: tx.id,
    createdAt: tx.createdAt.toISOString(),
  });
});

```

```

app.put("/api/transactions/:id", { schema: updateTransactionSchema }, async (request, reply) => {
  const { id } = request.params as { id: string };
  const userId = request.userId;
  const data = request.body as Record<string, unknown>;

```

-----

Liqing Accounting Software V1.0      LAST PAGE 23

=====

(50 source lines on this page)

```

  const existing = await prisma.transaction.findUnique({ where: { id } });
  if (!existing || existing.userId !== userId) {
    return reply.status(404).send({ error: "_____" });
  }

  const nextDirection = (data.direction as "expense" | "income" | undefined) ?
? existing.direction;
  const nextCategoryId = (data.categoryId as string | undefined) ?? existing.categoryId;
  const category = await findAccessibleCategory(userId, nextCategoryId);
  if (!category) {
    return reply.status(400).send({ error: "_____" });
  }
  if (category.type !== nextDirection) {

```

```

    return reply.status(400).send({ error: "-----" });
  }

  if (data.occurredAt) {
    data.occurredAt = new Date(data.occurredAt as string);
  }

  const updated = await prisma.transaction.update({
    where: { id },
    data,
  });

  return { transactionId: updated.id, updatedAt: updated.updatedAt.toISOString
() }
});

app.delete(
  "/api/transactions/:id",
  { schema: deleteTransactionSchema },
  async (request, reply) => {
    const { id } = request.params as { id: string };
    const userId = request.userId;

    const existing = await prisma.transaction.findUnique({ where: { id } });
    if (!existing || existing.userId !== userId) {
      return reply.status(404).send({ error: "-----" });
    }

    await prisma.transaction.delete({ where: { id } });
    return reply.status(204).send();
  },
);
}

export const registerSchema = {
  body: {
    type: "object",
    additionalProperties: false,
    required: ["username", "email", "displayName", "password"],
  },
  -----

```

Li Qing Accounting Software V1.0      LAST PAGE 24

=====  
 (50 source lines on this page)

```

    properties: {
      username: {
        type: "string",
        minLength: 3,
        maxLength: 20,
        pattern: "^[a-zA-Z0-9_]+$",
      },
      email: { type: "string", format: "email" },
      password: { type: "string", minLength: 6 },
      displayName: { type: "string", minLength: 1, maxLength: 20 },
      code: { type: "string", minLength: 6, maxLength: 6, pattern: "^[0-9]{6}$"
    },
  },
},
} as const;

export const loginSchema = {

```

```

    body: {
      type: "object",
      additionalProperties: false,
      required: ["identifier", "password"],
      properties: {
        identifier: { type: "string", minLength: 1, maxLength: 100 },
        password: { type: "string", minLength: 1 },
      },
    },
  },
} as const;

export const meSchema = {} as const;

export const sendCodeSchema = {
  body: {
    type: "object",
    additionalProperties: false,
    required: ["email"],
    properties: {
      email: { type: "string", format: "email" },
    },
  },
} as const;

export const verifyCodeSchema = {
  body: {
    type: "object",
    additionalProperties: false,
    required: ["email", "code"],
    properties: {
      email: { type: "string", format: "email" },
      code: { type: "string", minLength: 6, maxLength: 6, pattern: "^[0-9]{6}$" },
    },
  },
} as const;

```

-----

Liqing Accounting Software V1.0      LAST PAGE 25

=====

(50 source lines on this page)

```

} as const;
export const billsListSchema = {
  querystring: {
    type: "object",
    properties: {
      month: { type: "string", pattern: "^\\d{4}-\\d{2}$" },
      filterType: { type: "string", enum: ["all", "expense", "income"], default: "all" },
      query: { type: "string" },
      cursor: { type: "string" },
      pageSize: { type: "integer", minimum: 1, maximum: 100, default: 20 },
    },
  },
} as const;
const categoryTypeSchema = { type: "string", enum: ["expense", "income"] } as const;

export const listCategoriesSchema = {} as const;

export const createCategorySchema = {

```

```

    body: {
      type: "object",
      required: ["name", "icon", "type"],
      properties: {
        name: { type: "string", minLength: 1, maxLength: 20 },
        icon: { type: "string", minLength: 1, maxLength: 60 },
        type: categoryTypeSchema,
      },
    },
  },
} as const;

export const updateCategorySchema = {
  params: {
    type: "object",
    required: ["id"],
    properties: {
      id: { type: "string", minLength: 1 },
    },
  },
  body: {
    type: "object",
    minProperties: 1,
    properties: {
      name: { type: "string", minLength: 1, maxLength: 20 },
      icon: { type: "string", minLength: 1, maxLength: 60 },
      type: categoryTypeSchema,
    },
  },
} as const;

export const reorderCategoriesSchema = {
  body: {

```

-----

Liqing Accounting Software V1.0      LAST PAGE 26

=====

(50 source lines on this page)

```

    type: "object",
    required: ["type", "categoryIds"],
    properties: {
      type: categoryTypeSchema,
      categoryIds: {
        type: "array",
        minItems: 1,
        uniqueItems: true,
        items: { type: "string", minLength: 1 },
      },
    },
  },
} as const;

export const deleteCategorySchema = {
  params: {
    type: "object",
    required: ["id"],
    properties: {
      id: { type: "string", minLength: 1 },
    },
  },
} as const;

```

```

export const exportDataSchema = {
  querystring: {
    type: "object",
    properties: {
      startMonth: { type: "string", pattern: "^\\d{4}-\\d{2}$" },
      endMonth: { type: "string", pattern: "^\\d{4}-\\d{2}$" },
    },
  },
};
export const homeSummarySchema = {
  querystring: {
    type: "object",
    required: ["month"],
    properties: {
      month: { type: "string", pattern: "^\\d{4}-\\d{2}$" },
    },
  },
} as const;
export const statsOverviewSchema = {
  querystring: {
    type: "object",
    required: ["month"],
    properties: {
      month: { type: "string", pattern: "^\\d{4}-\\d{2}$" },
      metricType: { type: "string", enum: ["expense", "income"] },
    },
  },
},

```

-----

Liqing Accounting Software V1.0      LAST PAGE 27

=====

(50 source lines on this page)

```

} as const;
export const createTransactionSchema = {
  body: {
    type: "object",
    required: ["title", "amount", "direction", "categoryId", "occurredAt"],
    properties: {
      title: { type: "string", minLength: 1, maxLength: 100 },
      amount: { type: "number", exclusiveMinimum: 0 },
      direction: { type: "string", enum: ["expense", "income"] },
      categoryId: { type: "string", minLength: 1 },
      occurredAt: { type: "string", format: "date-time" },
      note: { type: "string", maxLength: 500 },
      merchant: { type: "string", maxLength: 100 },
    },
  },
} as const;

export const updateTransactionSchema = {
  params: {
    type: "object",
    required: ["id"],
    properties: {
      id: { type: "string" },
    },
  },
  body: {
    type: "object",
    properties: {

```

```

    title: { type: "string", minLength: 1, maxLength: 100 },
    amount: { type: "number", exclusiveMinimum: 0 },
    direction: { type: "string", enum: ["expense", "income"] },
    categoryId: { type: "string", minLength: 1 },
    occurredAt: { type: "string", format: "date-time" },
    note: { type: "string", maxLength: 500 },
    merchant: { type: "string", maxLength: 100 },
  },
},
} as const;

export const deleteTransactionSchema = {
  params: {
    type: "object",
    required: ["id"],
    properties: {
      id: { type: "string" },
    },
  },
},
} as const;
import "dotenv/config";
import { buildApp } from "./app.js";

```

-----

Liqing Accounting Software V1.0      LAST PAGE 28

=====

(50 source lines on this page)

```

const PORT = Number(process.env.PORT) || 3002;
const HOST = process.env.HOST || "0.0.0.0";

async function start() {
  const app = await buildApp();

  try {
    await app.listen({ port: PORT, host: HOST });
    app.log.info(`Server running at http://${HOST}:${PORT}`);
    app.log.info(`API docs at http://${HOST}:${PORT}/docs`);
  } catch (err) {
    app.log.error(err);
    process.exit(1);
  }
}

start();
import type { Decimal } from "@prisma/client/runtime/library";

/** _ month _ "2026-03" _ */
export function monthRange(month: string): { start: Date; end: Date } {
  const [year, mon] = month.split("-").map(Number);
  const start = new Date(Date.UTC(year, mon - 1, 1));
  const end = new Date(Date.UTC(year, mon, 1));
  return { start, end };
}

/** Decimal _ number */
export function toNumber(val: Decimal): number {
  return Number(val);
}
import type { PrismaClient, TransactionType } from "@prisma/client";

```

```

type CategoryTemplate = {
  id: string;
  name: string;
  icon: string;
  type: TransactionType;
  sort: number;
};

export const defaultCategoryTemplates: CategoryTemplate[] = [
  { id: "tpl_expense_dining", name: "__", icon: "a-068_yongcan", type: "expense", sort: 1 },
  { id: "tpl_expense_transit", name: "__", icon: "a-068_gongjiao", type: "expense", sort: 2 },
  { id: "tpl_expense_shopping", name: "__", icon: "a-068_gouwu", type: "expense", sort: 3 },
  { id: "tpl_expense_life", name: "__", icon: "a-068_shenghuo", type: "expense", sort: 4 },
  { id: "tpl_expense_utilities", name: "__", icon: "a-068_shuidian", type: "expense", sort: 5 },
  { id: "tpl_expense_fuel", name: "__", icon: "a-068_jiayou", type: "expense", sort: 6 },
  { id: "tpl_expense_travel", name: "__", icon: "a-068_lvyou", type: "expense", sort: 7 },

```

-----

Liqing Accounting Software V1.0      LAST PAGE 29

=====

(50 source lines on this page)

```

  { id: "tpl_expense_medical", name: "__", icon: "a-068_jiuyi", type: "expense", sort: 8 },
  { id: "tpl_income_salary", name: "__", icon: "a-068_gongzi", type: "income", sort: 1 },
  { id: "tpl_income_parttime", name: "__", icon: "a-068_jianzhi", type: "income", sort: 2 },
  { id: "tpl_income_finance", name: "__", icon: "a-068_licai", type: "income", sort: 3 },
  { id: "tpl_income_transfer", name: "__", icon: "a-068_zhuanzhang", type: "income", sort: 4 },
  { id: "tpl_income_reimbursement", name: "__", icon: "a-068_baoxiao", type: "income", sort: 5 },
];

```

```

export async function ensureGlobalCategoryTemplates(prisma: PrismaClient) {
  for (const template of defaultCategoryTemplates) {
    await prisma.category.upsert({
      where: { id: template.id },
      update: {
        name: template.name,
        icon: template.icon,
        type: template.type,
        sort: template.sort,
        userId: null,
      },
      create: {
        id: template.id,
        name: template.name,
        icon: template.icon,
        type: template.type,
        sort: template.sort,

```

```

        },
        {
            type: "category",
            data: {
                name: "category",
                icon: "category",
                type: "category",
                sort: "category",
                userId: null,
            },
        },
    ],
});

export async function ensureUserCategories(prisma: PrismaClient, userId: string)
{
    const userCategoryCount = await prisma.category.count({
        where: { userId },
    });
    if (userCategoryCount > 0) return;

    await ensureGlobalCategoryTemplates(prisma);

    const templates = await prisma.category.findMany({
        where: { userId: null },
        orderBy: [{ type: "asc" }, { sort: "asc" }],
    });

    if (templates.length == 0) return;

    await prisma.category.createMany({
        data: templates.map((template) => ({
            name: template.name,
            icon: template.icon,
        })),
    });
}

```

-----

Liqing Accounting Software V1.0      LAST PAGE 30

=====

(50 source lines on this page)

```

        type: template.type,
        sort: template.sort,
        userId,
    },
    {
        type: "category",
        data: {
            name: "category",
            icon: "category",
            type: "category",
            sort: "category",
            userId: null,
        },
    },
],
});

export async function resequenceUserCategories(
    prisma: PrismaClient,
    userId: string,
    type: TransactionType,
) {
    const categories = await prisma.category.findMany({
        where: { userId, type },
        orderBy: [{ sort: "asc" }, { id: "asc" }],
    });

    await prisma.$transaction(
        categories.map((category, index) =>
            prisma.category.update({
                where: { id: category.id },
                data: { sort: index + 1 },
            })
        ),
    );
}

import type { FastifyReply } from "fastify";

export class AppError extends Error {

```

```

    constructor(
      public statusCode: number,
      message: string,
    ) {
      super(message);
    }
  }

export function sendError(reply: FastifyReply, statusCode: number, message: string) {
  return reply.status(statusCode).send({ error: message });
}

import { PrismaClient } from "@prisma/client";

let prisma: PrismaClient;

export function getPrisma(): PrismaClient {
  if (!prisma) {
    prisma = new PrismaClient();
  }
  return prisma;
}

```

-----